

---

# Conditional Clifford-Steerable CNNs for PDE Modeling

---

Bálint László Szarvas<sup>\* 1</sup> Maksim Zhdanov<sup>\* 1</sup>

## Abstract

We introduce Conditional Clifford-Steerable CNNs (C-CSCNNs), a unified framework that incorporates equivariance to arbitrary pseudo-Euclidean groups and significantly improves the expressivity of standard CSCNNs. We show that the kernel basis of the standard formulation is incomplete, limiting model capacity. To address this, we augment the kernels with equivariant representations of the input feature field. We derive the equivariance constraint for these input-dependent kernels and show how it can be solved efficiently via implicit parameterization. We empirically validate on multiple PDE forecasting tasks, including fluid dynamics and relativistic electrodynamics, where our method consistently outperforms standard CSCNNs and performs on par with state-of-the-art baselines.

## 1. Introduction

Physical systems exhibit fundamental symmetries that constrain their governing equations (Wang, 2021). Any physically trustworthy model must therefore respect these symmetries and be consistent under the relevant transformations. When these transformations form a symmetry group, models that satisfy this consistency constraint are called *equivariant* (Bronstein et al., 2021). Equivariance has become a key inductive bias in critical applications where adherence to physical laws is required, such as drug and material design (Kovács et al., 2023) or robotics (Lin et al., 2023). By restricting the hypothesis class to functions that respect the symmetries of the problem, equivariant models need not learn these relations from data alone, which has been shown to improve data efficiency and often accelerate convergence during training (Brehmer et al., 2025; Weiler et al., 2023).

---

<sup>\*</sup>Equal contribution <sup>1</sup>University of Amsterdam, Amsterdam, The Netherlands. Correspondence to: Bálint László Szarvas <balint.szarvas@student.uva.nl>.

The deep learning community has developed multiple rich theoretical frameworks for constructing equivariant models (Nyholm et al., 2025). For convolutional neural networks (CNNs), the theory of steerable CNNs (Cohen & Welling, 2017) provides a general approach for incorporating equivariance to translations and transformations from an origin-preserving group  $G$ . A common example is the Euclidean group  $E(n)$ , where origin-preserving transformations are reflections and rotations that constitute the orthogonal group  $O(n)$ . The key idea behind steerable CNNs is to leverage the translation-equivariance of convolution and achieve  $G$ -equivariance by enforcing a  $G$ -induced constraint on the convolutional kernels. However, solving this constraint is not trivial and is usually involved. To alleviate this complexity, Zhdanov et al. (2023) proposed implicit parameterization using  $G$ -equivariant MLPs, with the resulting convolutions theoretically guaranteed to be  $G$ -equivariant.

Zhdanov et al. (2024) further generalized steerable CNNs to pseudo-Euclidean spaces, such as Minkowski spacetime, through Clifford-Steerable CNNs (CSCNNs). CSCNNs process feature fields by implementing  $G$ -steerable kernels implicitly via Clifford group equivariant neural networks, leveraging the correspondence between Clifford algebra and pseudo-Euclidean groups. The geometric expressivity of Clifford algebra and the data-efficient generalization ability of equivariant models combine to deliver strong performance on PDE forecasting tasks. However, the original formulation suffers from an inherent limitation: the incompleteness of its implicit kernels. Compared to theoretically derived kernels, certain bases are missing, which significantly limits the expressivity and performance of CSCNNs. Although the authors suggested that in theory the missing bases can be recovered through consecutive convolutions, this solution essentially doubles the necessary parameters, thus is inapplicable in practice.

The main contributions of this work are the following:

- We propose *Conditional Clifford Steerable CNNs (C-CSCNNs)*, which remedy the limited expressiveness of original CSCNNs by augmenting  $G$ -steerable kernels with auxiliary variables derived from the input feature fields.
- We mathematically derive the steerability constraint that conditional kernels must satisfy to maintain  $G$ -

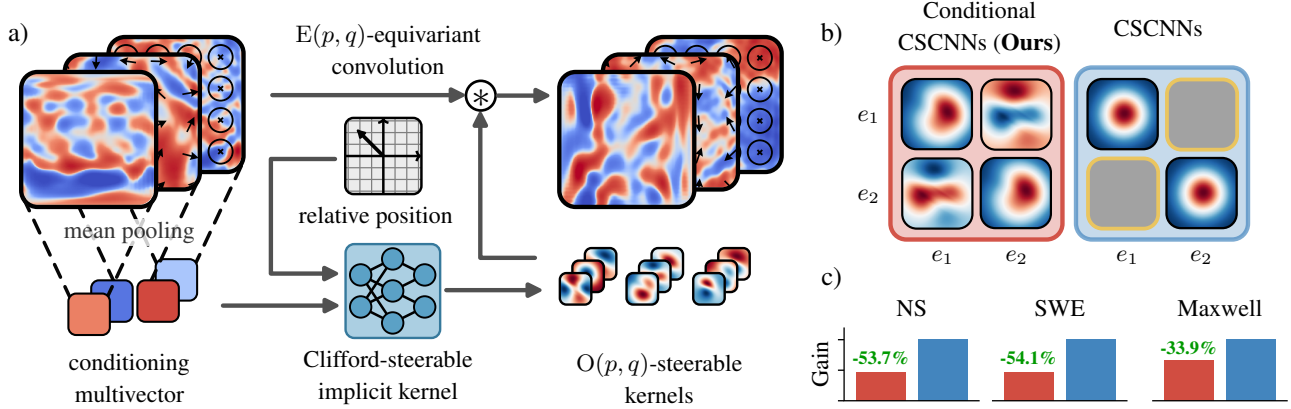


Figure 1. Conditional Clifford-Steerable CNNs use auxiliary information derived from the input feature field to condition the implicit kernel generating  $O(p, q)$ -steerable kernels (a). The interaction of the additional features with relative positions remedies limited expressivity of the original approach, yielding richer kernel basis (b) and, consequently, substantial performance gains (c).

equivariance and demonstrate how this constraint can be solved efficiently via implicit parameterization.

- We empirically validate the improved expressiveness on multiple PDE forecasting tasks, where conditional CSCNNs significantly outperform baseline CSCNNs and achieve performance on par with state-of-the-art methods.

## 2. Related Works

**Equivariance** There exist multiple ways of incorporating equivariance in neural networks. Equivariant convolutions can be categorized as regular (Cohen & Welling, 2016; Bekkers et al., 2018; 2024) or steerable group convolutions (Weiler & Cesa, 2019; Cesa et al., 2022). Regular group convolutions rely on discretizing the group  $G$  and convolving with rotated/transformed versions of the filters. While fast, this framework only achieves approximate equivariance due to discretization and is limited to compact groups. Steerable group convolutions employ analytically derived kernels that exactly satisfy the  $G$ -equivariance constraint. They require knowing the irreducible representation of  $G$  along with the Clebsch-Gordan coefficients and harmonic basis functions, all of which are group-specific. Another approach is canonicalization (Mondal et al., 2023; Kaba et al., 2023), which uses a non-equivariant model as the computational backbone but first transforms (canonicalizes) the input to a fixed frame of reference, ensuring that the output transforms consistently. While it demonstrates promising results, it requires designing a suitable canonicalization procedure, which is in general non-trivial.

**Clifford Algebra** Neural networks based on Clifford algebra were recently popularized by Brandstetter et al. (2023) and have since then gained popularity (Ruhe et al., 2023c; Brehmer et al., 2023) for their ability to natively handle geometric information. By encoding scalars, vectors, and

higher-order geometric objects such as oriented areas as elements of a single algebra, Clifford-based networks can learn interactions between features of different geometric types through the geometric product, enabling them to exploit the structure of the data more effectively (Brandstetter et al., 2023). Ruhe et al. (2023a) further demonstrated the connection between the Clifford algebra and equivariance, achieving equivariance to the pseudo-Euclidean group  $E(p, q)$ . Building on this work, Zhdanov et al. (2024) generalized the idea to a convolutional neural network that demonstrated particularly good performance in PDE modelling. Other applications include fluid dynamics (Pepe et al., 2025), relativistic physics (Spinner et al., 2024) and molecular generation (Liu et al., 2025).

**Implicit (continuous) kernels** Parameterizing convolutional kernels as learnable functions that map coordinates to kernel values was initially proposed by Schütt et al. (2017) to handle irregularly sampled molecular data, which later gained popularity in handling general point cloud data (Fuchs et al., 2020; Thomas et al., 2018). A subsequent line of work extended the concept to sequence modelling due to its ability to natively handle irregularly sampled data (Romero et al., 2022; Sitzmann et al., 2020) and capture long-range dependencies (Knigge et al., 2023), thus making them particularly effective in applications to text and audio. Multiple approaches also explored using continuous kernels for equivariant architectures, e.g. by relaxing the equivariance property (van der Ouderaa et al., 2022) or simplifying imposing it (Zhdanov et al., 2023).

## 3. Theoretical background

### 3.1. Steerable CNNs

We are interested in linear maps that operate on feature (vector) fields over pseudo-Euclidean space  $\mathbb{R}^{p,q}$ . A feature

field is a function  $f : \mathbb{R}^{p,q} \rightarrow W$  that assigns a feature vector  $f(x) \in W$  to each point  $x \in \mathbb{R}^{p,q}$ , where  $W$  is a vector space. The standard building block in deep learning for operating on such data is a convolutional neural layer, which we define as follows:

**Definition 3.1** (Convolution). Convolution maps between feature fields  $f_{\text{in}} : \mathbb{R}^{p,q} \rightarrow W_{\text{in}}$  and  $f_{\text{out}} : \mathbb{R}^{p,q} \rightarrow W_{\text{out}}$  via the integral transform

$$f_{\text{out}}(x) = L_K[f_{\text{in}}](x) := \int_{\mathbb{R}^{p,q}} d\mu(y) K(x-y) f_{\text{in}}(y). \quad (1)$$

where  $\mu$  is the Lebesgue measure on  $\mathbb{R}^{p,q}$ , and the kernel  $K : \mathbb{R}^{p,q} \rightarrow \text{Hom}(W_{\text{in}}, W_{\text{out}})$  is a function that assigns to each spatial offset  $z \in \mathbb{R}^{p,q}$  a linear map  $K(z) : W_{\text{in}} \rightarrow W_{\text{out}}$ .

We are now interested in enforcing the  $G$ -equivariance property on the convolutional operator. Before doing so, however, we must describe how feature fields are transformed by elements of  $G$ . Intuitively, transformations of the base space, which in our case is  $\mathbb{R}^{p,q}$ , imply corresponding transformations of the feature fields defined on them. To achieve this, we equip feature fields with a  $G$ -representation  $\rho$ , which, together with  $W$ , defines the geometric type of a feature vector  $(W, \rho)$  that fully describes the transformation law of a feature field under group action:

$$[\rho(g)f](x) := \rho(g)f(g^{-1}x) \quad \forall g \in G, x \in \mathbb{R}^{p,q}. \quad (2)$$

**Definition 3.2** (Equivariance). Consider arbitrary input and output feature fields of types  $(W_{\text{in}}, \rho_{\text{in}})$  and  $(W_{\text{out}}, \rho_{\text{out}})$ . The convolutional operator mapping between them is  $G$ -equivariant if and only if it satisfies

$$L_K[\rho_{\text{in}}(g)f_{\text{in}}] = \rho_{\text{out}}(g)L_K[f_{\text{in}}] \quad \forall g \in G \quad (3)$$

**Example 3.3.** The map in Def. 3.1 is translation equivariant. We can prove this by taking an arbitrary translation vector  $h$  and substituting  $\rho_{\text{in}}(h) = \rho_{\text{out}}(h) = T_h$ :

$$\begin{aligned} L_K[T_h f_{\text{in}}] &= \int_{\mathbb{R}^{p,q}} d\mu(y) K(x-y) f_{\text{in}}(y-h) \\ &= \int_{\mathbb{R}^{p,q}} d\mu(z) K((x-h)-z) f_{\text{in}}(z) = T_h L[f_{\text{in}}] \end{aligned}$$

where we used the substitution  $y-h \mapsto z$  and the translation invariance property of  $\mu$ .

The theory of steerable CNNs (Weiler et al., 2023) is concerned with imposing additional equivariance to an arbitrary group  $G$  by imposing the following constraint on the kernels:

**Theorem 3.4** (Steerable Convolution, (Weiler et al., 2023)). *The convolution integral in Def. 3.1 is  $G$ -equivariant if the kernel  $K$  satisfies the  $G$ -steerability constraint*

$$\begin{aligned} K(gx) &= \frac{1}{|\det(g)|} \rho_{\text{out}}(g) K(x) \rho_{\text{in}}(g)^{-1} \\ &= \rho_{\text{Hom}}(g) K(x) \quad \forall g \in G, x \in \mathbb{R}^{p,q} \end{aligned} \quad (4)$$

for feature fields of types  $(W_{\text{in}}, \rho_{\text{in}})$  and  $(W_{\text{out}}, \rho_{\text{out}})$ , and  $|\det(g)|$  is a volume scaling factor<sup>1</sup>

In general, one has to solve for the kernel analytically or numerically for each  $G$ , which is often non-trivial. Moreover, as we will see later, if we were to condition the kernel on an auxiliary variable, we would have to solve the constraint for each such case, which would be practically infeasible. A simple way to circumvent this complexity was suggested by Zhdanov et al. (2023), which boils down to parameterizing a kernel as a continuous function that returns the kernel matrix for each input. The following lemma defines the condition that such a function must satisfy for the resulting kernels to be steerable.

**Lemma 3.5** (Implicit parameterization, (Zhdanov et al., 2023)). *Assume that the linear map  $K(x) : W_{\text{in}} \rightarrow W_{\text{out}}$  has matrix representation  $[K(x)] \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}}}$ . Let  $\phi : \mathbb{R}^{p,q} \rightarrow \mathbb{R}^{c_{\text{out}} \times c_{\text{in}}}$  be a  $G$ -equivariant function satisfying*

$$\phi(gx) = (\rho_{\text{in}}(g) \otimes \rho_{\text{out}}(g)) \phi(x) \quad \forall g \in G, x \in \mathbb{R}^{p,q}. \quad (5)$$

*Then a kernel parameterized as  $[K(x)] = \phi(x)$  satisfies the equivariance constraint in Eq. 4.*

### 3.2. Clifford algebra and multivector fields

In previous works (Ruhe et al., 2023b; Zhdanov et al., 2024), it has been demonstrated that Clifford algebra  $\text{Cl}(\mathbb{R}^{p,q})$  is a representation space of the pseudo-orthogonal group  $\text{O}(p, q)$ . Let  $V$  be a vector space over  $\mathbb{R}^{p,q}$  equipped with a symmetric bilinear form  $\eta : V \times V \rightarrow \mathbb{R}^{p,q}$ . The Clifford algebra  $\text{Cl}(V, \eta)$  is the unitary, associative algebra satisfying

$$v \bullet v = \eta(v, v) \cdot \mathbf{1}_{\text{Cl}(V, \eta)}, \quad \forall v \in V \subset \text{Cl}(V, \eta) \quad (6)$$

where  $\bullet : \text{Cl}(V, \eta) \times \text{Cl}(V, \eta) \rightarrow \text{Cl}(V, \eta)$  is the *geometric product*, an  $\text{O}(p, q)$ -equivariant bilinear operation. Elements of the algebra are *multivectors*, constructed as linear combinations of geometric products of vectors  $v_1, \dots, v_n \in V$ . The algebra decomposes into *grade* subspaces  $\text{Cl}^{(k)}(V, \eta)$  for  $k = 0, \dots, d$ , where  $d = \dim V$ , with  $\dim \text{Cl}^{(k)}(V, \eta) = \binom{d}{k}$ . Grades  $k = 0$  and  $k = 1$  correspond to scalars and vectors, respectively, while

<sup>1</sup>For compact groups, e.g.  $\text{O}(2)$ , the factor is 1 and is usually omitted in literature.

$k \geq 2$  yields bivectors, trivectors and higher-grade elements representing oriented areas, volumes and their higher-dimensional analogues. Multivectors are associated with the orthogonal representation  $\rho_{\text{Cl}}$  and can thus be used as features of  $O(p, q)$ -equivariant networks (Ruhe et al., 2023b). For a comprehensive introduction to Clifford algebra, we refer to Appendix D of Zhdanov et al. (2024).

### 3.3. Clifford-Steerable CNNs

Building on the connection between  $\text{Cl}(\mathbb{R}^{p,q})$  and  $O(p, q)$ , implicit parameterization enables solving the  $O(p, q)$ -steerability constraint, allowing implementation of  $E(p, q)$ -equivariant convolutions (Zhdanov et al., 2024). CSCNNs use an  $O(p, q)$ -equivariant network for implicit parameterization of the kernels of a  $E(p, q)$ -equivariant convolution, which operates on  $c$ -dimensional multivector feature fields  $f : \mathbb{R}^{p,q} \rightarrow \text{Cl}(\mathbb{R}^{p,q})^c$  of type  $(W, \rho) = (\text{Cl}(\mathbb{R}^{p,q})^c, \rho_{\text{Cl}}^c)$ . Specifically, the implicit kernel  $K$  is a composition of two operators: kernel network  $\mathcal{K}$  that returns the multivector-valued matrix representation of the kernel

$$\mathcal{K} : \mathbb{R}^{p,q} \rightarrow \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{out}} \times c_{\text{in}}}, \quad (7)$$

and a *kernel head*  $H$  that turns the output of  $\mathcal{K}$  into a proper steerable kernel

$$H : \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{out}} \times c_{\text{in}}} \rightarrow \text{Hom}(\text{Cl}(\mathbb{R}^{p,q})^{c_{\text{in}}}, \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{out}}}). \quad (8)$$

by partially evaluating the geometric product between input and output feature fields:

$$K = H \circ \mathcal{K}, \quad K_n^k(x) = \sum_m \Lambda_{mn}^k w_{mn}^k \mathcal{K}(x)^{(m)} \quad (9)$$

where  $k$  is the input grade,  $n$  is the output grade,  $\Lambda_{mn}^k \in \{-1, 0, 1\}$  indicates how the geometric product between the grades  $m$  and  $n$  manifests in grade  $k$  of the result,  $w_{mn}^k$  is a learnable weight.

### 3.4. Incompleteness of the kernel basis of CSCNNs

Zhdanov et al. (2024) also showed that, when compared to the analytical solution for  $G = O(2) \equiv O(2, 0)$ , Clifford-steerable kernels miss degrees of freedom corresponding to angular frequency 2 for the vector-vector field interaction. Therefore, a single layer of CSCNNs is over-constrained by construction, which results in limited expressivity.

The source of the incompleteness lies in the input type of the kernel  $K$ , and how it is transformed throughout the kernel network  $\mathcal{K}$ . To demonstrate it, let us consider the case of  $O(2, 0)$ , for which the analytical solution is derived (Weiler & Cesa, 2019).

**Example 3.6.** The kernel  $K$  takes a single vector  $x \in \mathbb{R}^{2,0}$  that forms the grade-1 component of the input multivector.

Inside  $\mathcal{K}$ , the only transformations that are applied to the vector are (weighted) geometric products and linear grade-wise transformations. Using polar coordinates of  $\mathbb{R}^2 \equiv \mathbb{R}^{2,0}$ , i.e. a radius  $r \in \mathbb{R}_{\geq 0}$  and angle  $\phi \in S$ , it can be shown (see Appendix A.4 for details) that none of these operations allows angular information  $\phi$  to propagate to any grade other than grade-1:

$$\begin{aligned} \mathcal{K}(x)^{(0)} &= \mathbb{R}_0(r), \quad \mathcal{K}(x)^{(1)} = R_1(r)\kappa_1(\phi), \\ \mathcal{K}(x)^{(2)} &= 0 \quad \forall c_{\text{out}}, c_{\text{in}} \end{aligned}$$

where  $R_m, \kappa_m$  denote arbitrary nonlinear functions of  $r$  and  $\phi$ , respectively. We are interested specifically in the vector-vector interaction, as it is the part that is missing in the kernel basis. Let  $k = n = 1$ , then  $\Lambda_{01}^1 = \Lambda_{21}^1 = 1$ ,  $\Lambda_{11}^1 = 0$ , and the sum in Eq. 9 simplifies to

$$K_1^1(r, \phi) = \mathcal{K}(x)^{(0)} + 0 + 0 = \mathbb{R}_0(r) \quad (10)$$

Consequently, the resulting kernel represents scalar multiplication of the input vector with no angular dependence. In contrast, the complete analytical solution contains both frequency-0 and frequency-2 components, as predicted by the Clebsch-Gordan decomposition of  $O(2)$ -irrep tensor products (Lang & Weiler, 2021). Importantly, the frequency-2 component cannot be generated by the kernel network  $\mathcal{K}$  because its operations on single vector inputs restrict angular information to grade-1.

To re-iterate, the incompleteness, at least in the case of  $O(2, 0)$ , appears because the vector-vector interaction encoded by the implicit kernel collapses to a scalar value that does not contain angular information when only a single vector serves as input to the geometric product-based neural network. Since we cannot change how the geometric product works, our only option is to change the input to the kernel, which is what we propose in the following section.

## 4. Conditional Clifford Steerable CNNs

Instead of using a purely linear convolutional operator, we can introduce a dependency on the input feature field inside the convolutional kernel. This expands the input to a combination of multivectors, enabling the geometric product-based neural network to encode more expressive kernels and address the limitation of geometric product-based implicit kernels discussed above.

### 4.1. Conditional Steerable Convolution

Conditioning a convolutional kernel on the input yields the following non-linear operator<sup>2</sup>:

<sup>2</sup>For a comprehensive theoretical study of non-linear equivariant operators, see (Nyholm et al., 2025).

**Definition 4.1** (Conditional Convolution). Conditional convolution maps between feature fields<sup>3</sup>  $f : \mathbb{R}^{p,q} \rightarrow W_{\text{in}}$  and  $f_{\text{out}} : \mathbb{R}^{p,q} \rightarrow W_{\text{out}}$  via the integral transform

$$f_{\text{out}}(x) = L_{\hat{K}}[f](x) := \int_{\mathbb{R}^{p,q}} d\mu(y) \hat{K}(x-y, f(x), f(y)) f(y). \quad (11)$$

where  $\mu$  is the Lebesgue measure on  $\mathbb{R}^{p,q}$ , and the kernel  $\hat{K} : \mathbb{R}^{p,q} \times W_{\text{in}} \times W_{\text{in}} \rightarrow \text{Hom}(W_{\text{in}}, W_{\text{out}})$  is a function that assigns to every combination of spatial offset  $z \in \mathbb{R}^{p,q}$  and points  $x, y \in \mathbb{R}^{p,q}$  a linear map  $\hat{K}(z, f(x), f(y)) : W_{\text{in}} \rightarrow W_{\text{out}}$ .

Similarly to steerable CNNs, we enforce the  $G$ -equivariance property on the convolutional operator via the  $G$ -steerability constraint on the conditional kernel  $\hat{K}$ :

**Lemma 4.2** (Steerable Conditional Convolution). *The convolution integral 4.1 is  $G$ -equivariant if the kernel  $\hat{K}$  satisfies the following  $G$ -steerability constraint*

$$\begin{aligned} & \hat{K}(g(x-y), \rho_{\text{in}}(g)f(x), \rho_{\text{in}}(g)f(y)) \\ &= \rho_{\text{Hom}}(g) \hat{K}(x-y, f(x), f(y)), \quad \forall g \in G, x \in \mathbb{R}^{p,q}. \end{aligned} \quad (12)$$

for input and output feature field types  $(W_{\text{in}}, \rho_{\text{in}})$  and  $(W_{\text{out}}, \rho_{\text{out}})$ , respectively.

We provide the proof in Appendix A.1.

## 4.2. Clifford-Steerable Conditional Convolution

As in standard CSCNNs, we use implicit parameterization to achieve equivariance, but we need to adapt the kernel structure to accommodate the auxiliary information. The kernel head  $H$  is agnostic to the input of the kernel, furthermore, it is only the kernel network  $\hat{K}$  that we have to change to be compatible with conditional convolution. Let us define conditional kernel network as

$$\hat{K} : \mathbb{R}^{p,q} \times \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{in}}} \times \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{in}}} \rightarrow \text{Cl}(\mathbb{R}^{p,q})^{c_{\text{out}} \times c_{\text{in}}}. \quad (13)$$

which now takes additional multivector features as input. The function is equivariant by definition, as we use a  $O(p, q)$ -equivariant network to parameterize it. The equivariance of resulting Clifford-steerable conditional kernels  $\hat{K} := H \circ \hat{K}$  follows from the following lemma:

**Lemma 4.3** (Equivariance of conditional Clifford-steerable kernels). *Any Clifford-steerable conditional kernel  $\hat{K} = H \circ \hat{K}$  is  $O(p, q)$ -equivariant w.r.t  $\rho(g) = g$  and  $\rho_{\text{Hom}}$  as it satisfies Eq. 12.*

We provide the proof in Appendix A.2.

<sup>3</sup>In this section, we omit the subscript in  $f_{\text{in}}$  for clarity.

**Corollary 4.4** (Equivariance of Clifford-steerable conditional convolution). *Let  $\hat{K} = H \circ \hat{K}$  be a Clifford-steerable kernel. The corresponding convolution operator  $L_{\hat{K}}$  is then  $E(p, q)$ -equivariant:*

$$L_{\hat{K}}[\rho_{\text{in}}(g)f] = \rho_{\text{out}}(g)L_{\hat{K}}[f] \quad \forall g \in E(p, q). \quad (14)$$

## 4.3. Efficient Implementation via Template Matching

The theory admits arbitrary conditioning of the kernel  $\hat{K}$  on the input feature field. In the most general case, these might be features at the evaluation  $x$  and integration  $y$  points. This is similar to message-passing networks or point convolutions, where messages from neighbors to the origin node are typically conditional on the relative distance, as well as features of both sender and receiver nodes. However, such convolution is notoriously slow as it does not have any parameter sharing, the property that makes convolution efficient via template matching. We are furthermore looking for a trade-off between full expressivity (learning arbitrary interactions between two points) and efficiency (fully parallel computation).

To enable template matching, we must condition the kernel on a constant translation-invariant field derived from the input feature field. Let us introduce an operator  $T$  that does exactly that

$$\begin{aligned} T : L^2(\mathbb{R}^{p,q}, \text{Cl}(\mathbb{R}^{p,q})^c) &\rightarrow \text{Cl}(\mathbb{R}^{p,q})^c, \\ T[f](x) &= T[f] \quad \forall x \in \mathbb{R}^{p,q}, \end{aligned} \quad (15)$$

and whose output we use as the auxiliary input to Clifford-steerable conditional kernel.

Then the conditional convolution in Def. 4.1 takes the following form:

$$f_{\text{out}}(x) = L_{\hat{K}}^T[f](x) := \int_{\mathbb{R}^{p,q}} d\mu(y) \hat{K}(x-y, T[f]) f(y). \quad (16)$$

For the convolution to be equivariant, the implicit kernel that parameterizes it must be equivariant as well (Lemma 3.5), which in turn requires equivariance of the operator  $T$ :

**Proposition 4.5.** *The operator  $L_{\hat{K}}^T$  with a Clifford-Steerable conditional kernel  $\hat{K} = H \circ \hat{K}$  is  $E(p, q)$ -equivariant if the operator  $T$  is  $O(p, q)$ -equivariant, i.e.*

$$\begin{aligned} T[\rho(g)f] &= \rho(g)T[f] \\ \forall g \in O(p, q), f &\in L^2(\mathbb{R}^{p,q}, \text{Cl}(\mathbb{R}^{p,q})^c) \end{aligned} \quad (17)$$

We provide the proof in Appendix A.3.

The simplest choice for  $T$  is mean pooling whose equivariance is proven in (Weiler et al., 2023). Using mean pooling essentially replaces message-passing form of Eq. 11 with

a mean-field approximation where each point is influenced by the average behavior of all other points. While less expressive, the formulation nonetheless allows the kernels to adjust to global context, making it strictly more expressive than a standard convolution.

#### 4.4. Completeness of the kernel basis of Conditional CSCNNs

Let us now return to the case of  $O(2,0)$  and see how it will be handled in the suggested framework.

**Example 4.6** (Continued). Let us assume that the auxiliary input is a single multivector  $\zeta \in Cl(\mathbb{R}^{p,q})$ , whose grade-1 component does not coincide with the relative position vector  $x \in \mathbb{R}^{p,q}$ . It can be shown (see Appendix A.5), that after a single geometric product between  $\zeta$  and the embedded  $x$ , the kernel network output takes the following form

$$\mathcal{K}(x)^{(m)} = \mathbb{R}_m(r)\kappa_1(\phi) \quad \forall m, c_{\text{out}}, c_{\text{in}}$$

including the grade-0 case, which in the standard case is only a function of  $r$ . In other words, the angular information propagates through interaction with auxiliary multivectors. This, in turn, allows conditional implicit kernels to replicate the analytical solution as they are now able to generate higher frequency components.

Therefore, it is evident that using conditional kernels alleviates the incompleteness issue as the frequency-2 components are present in their basis. We formulate the result as a conjecture:

**Conjecture 4.7.** *The kernel basis of conditional Clifford-steerable CNNs is complete.*

We note that proving the conjecture requires knowing all irreducible representations of  $O(p, q)$  for each combination of  $p$  and  $q$ , as well as constructing an isomorphism between each irrep and multivector grades of  $Cl(p, q)$ . The task is highly non-trivial and somewhat contradictory to the nature of implicit kernels, which are designed to avoid deriving steerable kernels analytically. We therefore leave the proof for future work, and validate the claim empirically in the following section.

## 5. Experiments

To experimentally validate the improved expressivity of our implementation, we tested its performance on four well-established PDE modeling benchmarks, comparing it to the original CSCNN model **a total of 15 strong baselines across all tasks**. The experiments conducted were the 2-dimensional ( $\mathbb{R}^2$ ) *Navier-Stokes (NS)* equations, the 2-dimensional ( $\mathbb{R}^2$ ) *Shallow-water (SWE)* equations, the 3-dimensional ( $\mathbb{R}^3$ ) *Maxwell's (MW3)* equations, and the

relativistic 2-dimensional ( $\mathbb{R}^{1,2}$ ) *Maxwell's (MW2)* equations. Details on the datasets can be found in Appendix C.

We present our findings in subsection 5.3 organized by the what capabilities the tasks evaluate: first, we analyze *Data efficiency* (Table 1 and Figure 2) by comparing performance across different training set sizes while keeping parameter counts matched across baselines. Secondly, we investigate *Scaling* properties (Table 2) by evaluating differently sized architectures. Finally, we empirically verify the *Equivariance* of our approach.

### 5.1. Experimental setup

The goal of each experiment is to learn the dynamics of a system from numerical simulations. To enable flexible comparisons between different baselines, we treat timesteps as feature channels - i.e., the task becomes predicting future states (output channels) based on previous states (input channels) (Gupta & Brandstetter, 2022). For NS and SWE, the state is described by a vector velocity and a scalar pressure field. We evaluate two setups of SWE: one-step (SWE-1) and 5-step (SWE-5) ahead predictions. For MW3, the inputs are vector electric fields and bivector magnetic fields. In the relativistic MW2 task, the input is an electromagnetic field, which forms a bivector. In this setting time forms its own spacetime dimension, therefore, the task is to learn the mapping between two states, each with their respective spatial dimensions and a time dimension. This is the only experiment where the setup properly incorporates the time dimension into spacetime. Conditional CSCNNs (C-CSCNNs), similarly to regular CSCNNs, are well-equipped to fit this setting, while some of the baselines are unable to handle it by construction.

### 5.2. Implementation

Our model is built upon the ResNet (He et al., 2016) architecture, where we substitute the standard convolutional layers with our Conditional Clifford-Steerable convolutions. Since C-CSCNNs process multivector fields, we first embed the feature fields into their corresponding multivector grades. This is enabled by the natural isomorphisms  $\varepsilon^{(0)} : \mathbb{R} \xrightarrow{\sim} Cl(\mathbb{R}^{p,q})^{(0)}$ ,  $\varepsilon^{(1)} : \mathbb{R}^{p,q} \xrightarrow{\sim} Cl(\mathbb{R}^{p,q})^{(1)}$  between scalars, vectors and their corresponding  $k = 0$  and  $k = 1$  multivector grades. The multivector fields serve as the input to our convolutional layers and similarly, what the conditioning operator  $T$  is acting on, as illustrated in Figure 1. Through an ablation study shown in Appendix D.1, we found mean pooling to perform best as a simple equivariant conditioning operator  $T$ . Compared to the original CSCNNs, the only additional computational cost derives from computing the condition, which in the case of mean pooling is negligible. The implementation is done in JAX (Bradbury

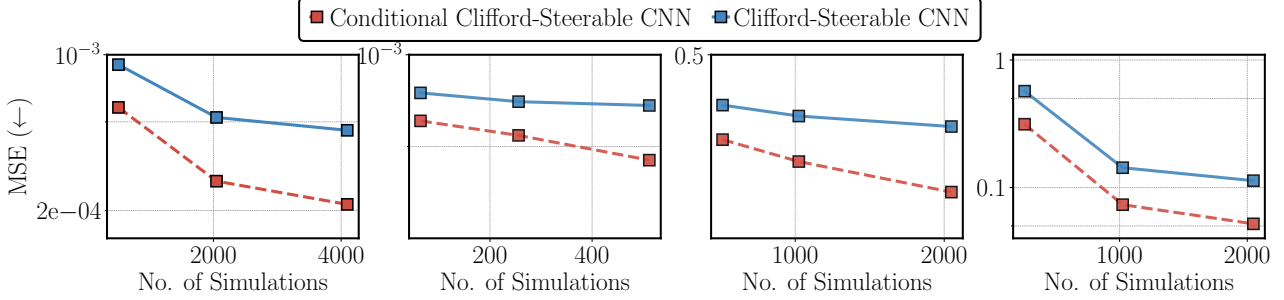


Figure 2. Mean squared errors for (1) Navier-Stokes  $\mathbb{R}^2$ , (2) Maxwell  $\mathbb{R}^3$ , (3) relativistic Maxwell  $\mathbb{R}^{1,2}$  and (4) Shallow-water equations  $\mathbb{R}^2$  1-step forecasting task simulation tasks as a function of the simulations included in the training dataset. Conditional CSCNNs outperform the standard CSCNN formulation, with its advantage increasing as more data is included in the training set.

et al., 2018) and Flax (Heek et al., 2024), leveraging the benefits of XLA compilation.

We compare Conditional CSCNNs against multiple families of strong neural solvers. This includes architecturally similar networks: the standard ResNet, the Clifford ResNet (Brandstetter et al., 2023), the  $O(n)$ -steerable CNN (Weiler & Cesa, 2019; Weiler et al., 2023) and the original Clifford-Steerable CNN (Zhdanov et al., 2024). Additionally, we include Fourier Neural Operator (FNO) (Li et al., 2021), its  $D_4 < O(2)$  equivariant counterpart G-FNO (Helwig et al., 2023) and the U-shaped neural operator UNO (Rahman et al., 2023). Finally, we evaluate against state-of-the-art neural PDE solvers: Transolver (Wu et al., 2024a), CViT (Wang et al., 2025), the Swin-Transformer (Liu et al., 2021), the latent-space solver LSM (Wu et al., 2023) and the classic U-net (Ronneberger et al., 2015). In each experiment except SWE-5, the parameter counts were matched to ensure fair comparison between the models. In SWE-5 however, we compared architectures of different sizes. For this, we take baselines from (Gupta & Brandstetter, 2022) and (Wang et al., 2025), which includes different sizes of CViT (Wang et al., 2025) and the Dilated ResNet (Yu et al., 2017), with two of its variants U-Net<sub>att</sub> (Gupta & Brandstetter, 2022) and U-F2Net (Gupta & Brandstetter, 2022). More details on the tasks and the implementations can be found in Appendix C and B.

### 5.3. Results

**Data efficiency** In the NS, SWE-1, MW3, and MW2 tasks, we investigate data efficiency by varying the number of trajectories included in the training set. As shown in Figure 2, owing to the recovered kernel bases, C-CSCNNs are able to leverage training data significantly better than standard CSCNNs. This improved expressivity makes them comparable to state-of-the-art baselines, as shown in Table 1. C-CSCNNs outperform all baselines on fluid dynamics tasks (NS, SWE-1) and achieve second-best performance on electrodynamics tasks (MW3, MW2) behind only Transolver. This suggests that the equivariance bias of C-CSCNNs provides a deci-

sive advantage in more local dynamics such as fluid flow, while richer conditioning operators may be needed to match attention in more non-local settings like electrodynamics.

| NS                        | $N = 512$       | $N = 2048$      | $N = 4096$      |
|---------------------------|-----------------|-----------------|-----------------|
| C-CSCNN (Ours)            | <b>5.80e-04</b> | <b>2.70e-04</b> | <b>1.80e-04</b> |
| Transolver                | 7.10e-04        | 4.10e-04        | 3.40e-04        |
| CSCNN                     | 9.00e-04        | 5.30e-04        | 4.50e-04        |
| Clifford-Steerable ResNet | 1.21e-03        | 9.14e-04        | 8.80e-04        |
| CViT                      | 2.15e-03        | 1.01e-03        | 9.45e-04        |
| SWE-1                     | $N = 256$       | $N = 1024$      | $N = 2048$      |
| C-CSCNN (Ours)            | <b>0.3137</b>   | <b>0.0734</b>   | <b>0.0519</b>   |
| Transolver                | <u>0.4515</u>   | <u>0.0903</u>   | <u>0.0594</u>   |
| CSCNN                     | 0.5699          | 0.1431          | 0.1132          |
| CViT                      | 0.7129          | 0.1974          | 0.1259          |
| Swin-Tr.                  | 0.8197          | 0.2073          | 0.1503          |
| MW3                       | $N = 64$        | $N = 256$       | $N = 512$       |
| C-CSCNN (Ours)            | 6.07e-04        | 5.44e-04        | 4.51e-04        |
| Transolver                | <b>4.96e-04</b> | <b>4.24e-04</b> | <b>3.99e-04</b> |
| CSCNN                     | 7.50e-04        | 7.02e-04        | 6.82e-04        |
| CViT                      | 1.28e-03        | 1.08e-03        | 9.65e-04        |
| ResNet                    | 3.34e-03        | 1.90e-03        | 1.67e-03        |
| MW2                       | $N = 512$       | $N = 1024$      | $N = 2048$      |
| C-CSCNN (Ours)            | <u>0.3273</u>   | <u>0.2934</u>   | <u>0.2519</u>   |
| Transolver                | <b>0.26342</b>  | <b>0.22149</b>  | <b>0.17034</b>  |
| CSCNN                     | 0.3889          | 0.3682          | 0.3494          |
| CViT                      | 0.65954         | 0.48625         | 0.45295         |
| ResNet                    | 0.8081          | 0.7661          | 0.6653          |

Table 1. MSE for the top 5 best performing baselines per task and the number of simulations included in their training dataset (denoted by  $N$ ). For each task and dataset size, the lowest MSE is bolded (lower is better) and the second-lowest is underlined. On NS and SWE-1, C-CSCNNs achieve the lowest MSE for all  $N$ . For MW2 and MW3, it achieves second lowest MSE behind Transolver. The fully complete results including all baselines are shown in Table 4 of Appendix D.

**Scaling** For the SWE-5 benchmark, we probed the scaling properties of our approach. Table 2 reports the relative  $L^2$  error for the 5 step predictions (see Fig.3 for a rollout example). In the small model regime (around 10M), C-CSCNNs significantly outperform even state-of-the-art models, such

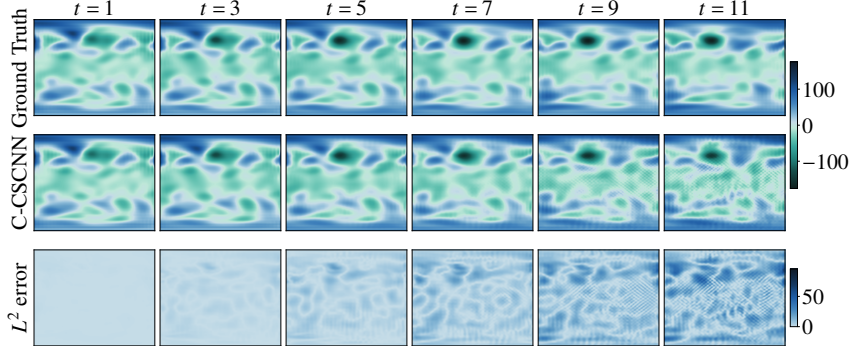


Figure 3. Relative  $L^2$  error of conditioned CSCNNs on the shallow water equations task at different timesteps of a simulation. The trajectories were obtained by autoregressive rollout of the model. Results are shown for component  $u$  of the wind velocity field.

as CViT. We expect that the scalability of C-CSCNNs can be significantly improved by introducing more complex and expressive conditioning operators, which higher parameter models can better leverage. Nevertheless, the simple mean pooling setup already demonstrates potential, as our scaled variant (55M) performs on par with leading approaches and exceed models significantly larger (such as FNO with 270M and UNO with 440M params).

Table 2. 5-step rollout performance on the shallow-water equations task by Relative  $L^2$  error (lower is better.) The results for baselines were taken from (Wang et al., 2025)

| Model                | #Params | Rel. $L^2$ |
|----------------------|---------|------------|
| DilResNet            | 4.2M    | 13.20%     |
| U-Net <sub>att</sub> | 148M    | 5.68%      |
| FNO                  | 268M    | 3.97%      |
| U-F2Net              | 344M    | 1.89%      |
| UNO                  | 440M    | 3.79%      |
| CViT-S               | 13M     | 4.47%      |
| CViT-B               | 30M     | 2.69%      |
| CViT-L               | 92M     | 1.56%      |
| C-CSCNN (Small)      | 10M     | 3.51%      |
| C-CSCNN (Large)      | 55M     | 2.94%      |

**Equivariance** To support our theoretical claims in Proposition 4.5, we evaluated the equivariance of our conditional convolutional layers against the original Clifford-Steerable convolutions. For this, we measure the relative  $E(2)$  equivariance error

$$\varepsilon_{\text{eq}}(L, f, g) = \frac{\|L[\rho(g)f] - \rho(g)L[f]\|}{\|L[\rho(g)f] + \|\rho(g)L[f]\|} \quad (18)$$

on single layer convolutions using the NS dataset. The original Clifford-Steerable convolution attains a mean relative equivariance error of  $2.4 \times 10^{-7}$ , while the conditional Clifford-Steerable convolution attains  $3.4 \times 10^{-7}$ . Up to discretisation error and numerical artifacts, these values are of the same order, providing an experimental validation of our equivariance proof.

## 6. Conclusion

We introduced Conditional Clifford-Steerable CNNs, a generalization of Clifford-Steerable CNNs that addresses the limited expressivity of the original framework. Our approach augments the implicit parametrization of Clifford-Steerable kernels with auxiliary variables derived from the input, effectively making the convolution operation non-linear. We derived the general constraint on such kernels necessary to maintain  $E(p, q)$ -equivariance and proved that our approach satisfies it. We theoretically showed that conditional kernels restore the completeness of the kernel basis for  $O(2, 0)$ , and provided empirical evidence that this holds more broadly. Through experimental evaluation on various PDE forecasting tasks, we demonstrated that our approach outperforms standard CSCNNs while achieving strong overall performance compared to state-of-the-art methods.

A key advantage of Conditional-CSCNNs is that they enforce equivariance to symmetry transformations of  $E(p, q)$  and any of its subgroups implicitly, making them flexible and particularly suitable whenever such symmetries are informative. While this work focuses on PDE learning, the framework is not restricted to these. Many problems on (pseudo-)Euclidean domains can be embedded into Clifford-algebra representation to better capture geometric relationships between features and exploit the inductive bias of equivariance. This suggests broad applicability to tasks such as 3D pose, camera rays or surface normal estimation in fields like optimal control (L  w & Calinon, 2023), robotics (Lin et al., 2023; L  w et al., 2024), computer vision (Chen et al., 2021) and physics (Kasieczka et al., 2017).

## Limitations

- While we prove completeness for  $O(2, 0)$ , the conjecture for arbitrary  $O(p, q)$  remains open. Our empirical improvements across multiple symmetry groups suggest a formal proof of the general case is a promising, albeit challenging direction.
- Our approach investigated simple, proof of concept

conditioning operators, though more complex problems likely necessitate richer operators.

- On discretized grids and finite domains, exact equivariance is not strictly achievable in practice. This is a general limitation of equivariant methods on discrete data, not specific to our approach, and the resulting errors are negligible.
- Conditional CSCNNs inherit some limitations of standard CSCNNs: there exist more general field types than multivectors and grades may not always be irreducible representations.

**Future work** The most promising avenue for future work is exploring different choices of the conditional operator. In this work, we already observed that by using simple operators such as mean or max pooling, we are able to achieve performance comparable to state-of-the-art approaches. Potential options include different types of learnable pooling or less restrictive forms of weight sharing, e.g. conditioning on finer yet still coarse regions of the domain. This would render the approach closer to hierarchical methods such as fast multipole method (Carrier et al., 1988) which recently found application in deep learning (Wang, 2023; Wu et al., 2024b; Zhdanov et al., 2025). Additionally, our current implementation uses a simple ResNet backbone. Using a stronger architecture such as ConvNeXt (Woo et al., 2023), which is competitive with Transformers, could further improve results.

## Impact Statement

This work improves neural surrogates for physical simulation. More accurate models can accelerate scientific discovery in fluid dynamics, materials design and climate modeling. More data-efficient models reduce the computational and environmental cost of training. By encoding known physical symmetries, our approach yields predictions that are more consistent with known physical laws. Beyond scientific modeling, the framework may benefit other domains involving geometric structure, such as robotics, computer vision, and optimal control. We do not foresee direct negative societal consequences of this work.

## References

- Bekkers, E. J., Lafarge, M. W., Veta, M., Eppenhof, K. A. J., Pluim, J. P. W., and Duits, R. Roto-translation covariant convolutional networks for medical image analysis. In *Medical Image Computing and Computer Assisted Intervention - MICCAI*, 2018.
- Bekkers, E. J., Vadgama, S. P., Hesselink, R., van der Linden, P. A., and Romero, D. W. Fast, expressive se(n) equivariant networks through weight-sharing in position-orientation space. In *The Twelfth International Conference on Learning Representations, ICLR*, 2024.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Brandstetter, J., van den Berg, R., Welling, M., and Gupta, J. K. Clifford neural layers for PDE modeling. In *The Eleventh International Conference on Learning Representations, ICLR*, 2023.
- Brehmer, J., de Haan, P., Behrends, S., and Cohen, T. S. Geometric algebra transformer. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.
- Brehmer, J., Behrends, S., de Haan, P., and Cohen, T. Does equivariance matter at scale? *Trans. Mach. Learn. Res.*, 2025, 2025. URL <https://openreview.net/forum?id=wilNute8Tn>.
- Bronstein, M. M., Bruna, J., Cohen, T., and Velickovic, P. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *CoRR*, abs/2104.13478, 2021. URL <https://arxiv.org/abs/2104.13478>.
- Carrier, J., Greengard, L., and Rokhlin, V. A fast adaptive multipole algorithm for particle simulations. *SIAM Journal on Scientific and Statistical Computing*, 9(4):669–686, 1988. doi: 10.1137/0909044.
- Cesa, G., Lang, L., and Weiler, M. A Program to Build E(N)-Equivariant Steerable CNNs. In *International Conference on Learning Representations (ICLR)*, 2022.
- Chen, H., Liu, S., Chen, W., Li, H., and Hill, R. Equivariant point network for 3d point cloud analysis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 14514–14523, 2021.
- Cohen, T. and Welling, M. Group equivariant convolutional networks. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.
- Cohen, T. S. and Welling, M. Steerable cnns. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=rJQKYt51l>.
- Filipovich, M. J. and Hughes, S. Pycharge: An open-source python package for self-consistent electrodynamics simulations of lorentz oscillators and moving point charges. *Comput. Phys. Commun.*, 274:108291, 2022.

- doi: 10.1016/J.CPC.2022.108291. URL <https://doi.org/10.1016/j.cpc.2022.108291>.
- Fuchs, F., Worrall, D. E., Fischer, V., and Welling, M. Se(3)-transformers: 3d roto-translation equivariant attention networks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020*, 2020.
- Gupta, J. K. and Brandstetter, J. Towards multi-spatiotemporal-scale generalized PDE modeling. *CoRR*, abs/2209.15616, 2022. doi: 10.48550/ARXIV.2209.15616. URL <https://doi.org/10.48550/arXiv.2209.15616>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, 2016.
- Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., and van Zee, M. Flax: A neural network library and ecosystem for JAX, 2024. URL <http://github.com/google/flax>.
- Helwig, J., Zhang, X., Fu, C., Kurtin, J., Wojtowysch, S., and Ji, S. Group equivariant fourier neural operators for partial differential equations. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org, 2023.
- Holl, P. and Thuerey, N.  $\Phi_{\text{flow}}$  (PhiFlow): Differentiable simulations for pytorch, tensorflow and jax. In *International Conference on Machine Learning*. PMLR, 2024.
- Kaba, S., Mondal, A. K., Zhang, Y., Bengio, Y., and Ravanbakhsh, S. Equivariance with learned canonicalization functions. In *International Conference on Machine Learning, ICML*, 2023.
- Kasieczka, G., Plehn, T., Russell, M., and Schell, T. Deep-learning top taggers or the end of qcd? *Journal of High Energy Physics*, 2017(5):1–22, 2017.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- Klöwer, M., Gelbrecht, M., Hotta, D., Willmert, J., Silvestri, S., Wagner, G. L., White, A., Hatfield, S., Kimpson, T., Constantinou, N. C., and Hill, C. Speedy-Weather.jl: Reinventing atmospheric general circulation models towards interactivity and extensibility. *Journal of Open Source Software*, 9(98):6323, 2024. doi: 10.21105/joss.06323. URL <https://doi.org/10.21105/joss.06323>.
- Knigge, D. M., Romero, D. W., Gu, A., Bekkers, E. J., Gavves, E., Tomczak, J. M., Hoogendoorn, M., and Sonke, J.-J. Modelling long range dependencies in n-d: From task-specific to a general purpose cnn. *International Conference on Learning Representations*, 2023.
- Kovács, D. P., Moore, J. H., Browning, N. J., Batatia, I., Horton, J. T., Kapil, V., Witt, W. C., Magdău, I.-B., Cole, D. J., and Csányi, G. Mace-off23: Transferable machine learning force fields for organic molecules, 2023.
- Lang, L. and Weiler, M. A wigner-eckart theorem for group equivariant convolution kernels. In *International Conference on Learning Representations (ICLR)*, 2021.
- Li, Z., Kovachki, N. B., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=c8P9NQVtmnO>.
- Lin, C. E., Song, J., Zhang, R., Zhu, M., and Ghaffari, M. Se (3)-equivariant point cloud-based place recognition. In *Conference on Robot Learning*, pp. 1520–1530. PMLR, 2023.
- Liu, C., Vadgama, S., Ruhe, D., Bekkers, E. J., and Forré, P. Clifford group equivariant diffusion models for 3d molecular generation. *CoRR*, abs/2504.15773, 2025.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*, pp. 9992–10002. IEEE, 2021. doi: 10.1109/ICCV48922.2021.00986. URL <https://doi.org/10.1109/ICCV48922.2021.00986>.
- Löw, T. and Calinon, S. Geometric algebra for optimal control with applications in manipulation tasks. *IEEE Transactions on Robotics*, 39(5):3586–3600, 2023.
- Löw, T., Abbet, P., and Calinon, S. Gafro: geometric algebra for robotics. *IEEE Robotics & Automation Magazine*, 2024.
- Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., Rathnayake, T., Vig, S., Granger, B. E., Muller, R. P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., Curry, M. J., Terrel, A. R., Roučka, v., Saboo, A., Fernando, I., Kulal, S., Cimrman, R., and Scopatz, A. SymPy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi: 10.7717/peerj-cs.103. URL <https://doi.org/10.7717/peerj-cs.103>.

- Mondal, A. K., Panigrahi, S. S., Kaba, O., Mudumba, S., and Ravanbakhsh, S. Equivariant adaptation of large pretrained models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023.
- Nyholm, E., Carlsson, O., Weiler, M., and Persson, D. Equivariant non-linear maps for neural networks on homogeneous spaces. *CoRR*, abs/2504.20974, 2025.
- Pepe, A., Montanari, M., and Lasenby, J. Fengbo: a clifford neural operator pipeline for 3d pdes in computational fluid dynamics. In *The Thirteenth International Conference on Learning Representations, ICLR*, 2025.
- Rahman, M. A., Ross, Z. E., and Azizzadenesheli, K. U-NO: u-shaped neural operators. *Trans. Mach. Learn. Res.*, 2023, 2023. URL <https://openreview.net/forum?id=j3oQF9coJd>.
- Romero, D. W., Kuzina, A., Bekkers, E. J., Tomczak, J. M., and Hoogendoorn, M. Ckconv: Continuous kernel convolution for sequential data. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=8FhxBtXSl0>.
- Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. In Navab, N., Hornegger, J., III, W. M. W., and Frangi, A. F. (eds.), *Medical Image Computing and Computer-Assisted Intervention - MICCAI 2015 - 18th International Conference Munich, Germany, October 5 - 9, 2015, Proceedings, Part III*, volume 9351 of *Lecture Notes in Computer Science*, pp. 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4\_28. URL [https://doi.org/10.1007/978-3-319-24574-4\\_28](https://doi.org/10.1007/978-3-319-24574-4_28).
- Ruhe, D., Brandstetter, J., and Forré, P. Clifford group equivariant neural networks. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, 2023a.
- Ruhe, D., Brandstetter, J., and Forré, P. Clifford group equivariant neural networks. *Advances in Neural Information Processing Systems*, 2023b.
- Ruhe, D., Gupta, J. K., Keninck, S. D., Welling, M., and Brandstetter, J. Geometric clifford algebra networks. In *International Conference on Machine Learning, ICML*, 2023c.
- Schütt, K., Kindermans, P., Felix, H. E. S., Chmiela, S., Tkatchenko, A., and Müller, K. Schnet: A continuous-filter convolutional neural network for modeling quantum interactions. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, 2017.
- Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., and Wetzstein, G. Implicit neural representations with periodic activation functions. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- Spinner, J., Bresó, V., de Haan, P., Plehn, T., Thaler, J., and Brehmer, J. Lorentz-equivariant geometric algebra transformers for high-energy physics. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, 2024.
- Thomas, N., Smidt, T. E., Kearnes, S., Yang, L., Li, L., Kohlhoff, K., and Riley, P. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds. *CoRR*, abs/1802.08219, 2018. URL <http://arxiv.org/abs/1802.08219>.
- van der Ouderaa, T. F. A., Romero, D. W., and van der Wilk, M. Relaxing equivariance constraints with non-stationary continuous filters. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Wang, P.-S. Octformer: Octree-based transformers for 3D point clouds. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4), 2023.
- Wang, R. Physics-guided deep learning for dynamical systems: A survey. *CoRR*, abs/2107.01272, 2021. URL <https://arxiv.org/abs/2107.01272>.
- Wang, R., Walters, R., and Yu, R. Incorporating symmetry into deep dynamics models for improved generalization. In *International Conference on Learning Representations*, 2021. URL [https://openreview.net/forum?id=wta\\_8Hx2KD](https://openreview.net/forum?id=wta_8Hx2KD).
- Wang, S., Seidman, J. H., Sankaran, S., Wang, H., Pappas, G. J., and Perdikaris, P. Cvit: Continuous vision transformer for operator learning. In *The Thirteenth International Conference on Learning Representations, ICLR*, 2025.

Weiler, M. and Cesa, G. General e(2)-equivariant steerable cnns. *Advances in Neural Information Processing Systems*, 2019.

Weiler, M., Forré, P., Verlinde, E., and Welling, M. Equivariant and coordinate independent convolutional networks. *A Gauge Field Theory of Neural Networks*, pp. 110, 2023.

Woo, S., Debnath, S., Hu, R., Chen, X., Liu, Z., Kweon, I. S., and Xie, S. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 16133–16142, 2023.

Wu, H., Hu, T., Luo, H., Wang, J., and Long, M. Solving high-dimensional pdes with latent spectral models. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 37417–37438. PMLR, 2023. URL <https://proceedings.mlr.press/v202/wu23f.html>.

Wu, H., Luo, H., Wang, H., Wang, J., and Long, M. Transolver: A fast transformer solver for pdes on general geometries. In *International Conference on Machine Learning*, 2024a.

Wu, X., Jiang, L., Wang, P., Liu, Z., Liu, X., Qiao, Y., Ouyang, W., He, T., and Zhao, H. Point transformer V3: simpler, faster, stronger. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024b.

Yu, F., Koltun, V., and Funkhouser, T. A. Dilated residual networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pp. 636–644. IEEE Computer Society, 2017. doi: 10.1109/CVPR.2017.75. URL <https://doi.org/10.1109/CVPR.2017.75>.

Zhdanov, M. Flash clifford: Hardware-efficient implementation of clifford algebra neural networks, 2025. URL <https://github.com/maxxxzdn/flash-clifford>.

Zhdanov, M., Hoffmann, N., and Cesa, G. Implicit convolutional kernels for steerable cnns. *Advances in Neural Information Processing Systems*, 2023.

Zhdanov, M., Ruhe, D., Weiler, M., Lucic, A., Brandstetter, J., and Forré, P. Clifford-steerable convolutional neural networks. *Forty-first International Conference on Machine Learning, ICML*, 2024.

Zhdanov, M., Welling, M., and van de Meent, J.-W. Erwin: A tree-based hierarchical transformer for large-scale physical systems. In *International Conference on Machine Learning (ICML)*, 2025.

## A. Proofs

### A.1. Proof of Lemma 4.2

*Proof.* A conditional convolution given in Def. 4.1 is  $G$ -equivariant if it satisfies

$$L_{\hat{K}}[\rho_{\text{in}}(g)f] = \rho_{\text{out}}(g)L_{\hat{K}}[f] \quad \forall g \in G. \quad (19)$$

Let us expand the left-hand side:

$$\begin{aligned} L_{\hat{K}}[\rho_{\text{in}}(g)f] &= \int_{\mathbb{R}^{p,q}} d\mu(y) \hat{K}(x - y, \rho_{\text{in}}(g)f(g^{-1}x), \\ &\quad \rho_{\text{in}}(g)f(g^{-1}y)) \rho_{\text{in}}(g)f(g^{-1}y) \end{aligned} \quad (20)$$

$$= |\det(g)| \int_{\mathbb{R}^{p,q}} d\mu(y) \hat{K}(x - gy, \rho_{\text{in}}(g)f(g^{-1}x), \rho_{\text{in}}(g)f(y)) \rho_{\text{in}}(g)f(y) \quad (21)$$

where we used the substitution  $g^{-1}y \mapsto y$ .

Writing out the right-hand side yields

$$\begin{aligned} \rho_{\text{out}}(g)L_{\hat{K}}[f] &= \int_{\mathbb{R}^{p,q}} d\mu(y) \rho_{\text{out}}(g)\hat{K}(g^{-1}x - y, \\ &\quad f(g^{-1}x), f(y))f(y) \end{aligned} \quad (22)$$

The expressions agree for any  $g \in G$  and feature map  $f \in L^2(\mathbb{R}^{p,q}, W)$  if and only if

$$\begin{aligned} |\det(g)|\hat{K}(x - gy, \rho_{\text{in}}(g)f(g^{-1}x), \rho_{\text{in}}(g)f(y)) \rho_{\text{in}}(g) \\ = \rho_{\text{out}}(g)\hat{K}(g^{-1}x - y, f(g^{-1}x), f(y)). \end{aligned} \quad (23)$$

After the substitution  $g^{-1}x \mapsto x$ , we obtain the constraint:

$$\begin{aligned} |\det(g)|\hat{K}(gx - gy, \rho_{\text{in}}(g)f(x), \rho_{\text{in}}(g)f(y)) \rho_{\text{in}}(g) \\ = \rho_{\text{out}}(g)\hat{K}(x - y, f(x), f(y)), \end{aligned} \quad (24)$$

which is equivalent to

$$\hat{K}(g(x - y), \rho_i(g)f(x), \rho_i(g)f(y)) \quad (25)$$

$$= \frac{1}{|\det(g)|} \rho_o(g)\hat{K}(x - y, f(x), f(y))\rho_i(g)^{-1} \quad (26)$$

$$= \rho_{\text{Hom}}(g)\hat{K}(x - y, f(x), f(y)). \quad (27)$$

□

### A.2. Proof of Lemma 4.3

*Proof.* A Clifford-steerable conditional kernel is a composition of kernel network  $\hat{K}$  and kernel head  $H$ . The former is equivariant by definition, and the equivariance of the latter

is proven in (Zhdanov et al., 2024). We can then write

$$\hat{K}(g(x-y), \rho_{\text{Cl}}^{\text{cin}}(g)f(x)\rho_{\text{Cl}}^{\text{cin}}(g)f(y)) \quad (28)$$

$$= H\left(\hat{K}(g(x-y), \rho_{\text{Cl}}^{\text{cin}}(g)f(x)\rho_{\text{Cl}}^{\text{cin}}(g)f(y))\right) \quad (29)$$

$$= H\left(\rho_{\text{Cl}}^{\text{c}_{\text{out}} \times \text{c}_{\text{in}}}(g)\hat{K}(x-y, f(x), f(y))\right) \quad (30)$$

$$= \rho_{\text{Hom}}(g)H\left(\hat{K}(x-y, f(x), f(y))\right) \quad (31)$$

$$= \rho_{\text{Hom}}(g)\hat{K}(x-y, f(x), f(y)) \quad (32)$$

□

### A.3. Proof of Proposition 4.5

*Proof.* By the definition of the kernel network  $\hat{K}$ , it satisfies the equivariance constraint

$$\hat{K}(gz, \rho(g)w) = (\rho(g) \otimes \rho(g))\hat{K}(z, w) \quad (33)$$

$$\forall g \in O(p, q), z \in \mathbb{R}^{p,q}, w \in \text{Cl}(\mathbb{R}^{p,q})^c. \quad (34)$$

Furthermore, the property is satisfied for  $w = T[f]$ :

$$\hat{K}(gz, \rho(g)T[f]) = (\rho(g) \otimes \rho(g))\hat{K}(z, T[f]) \quad (35)$$

On the other hand, for the kernel to be steerable, we require:

$$\hat{K}(gz, T[\rho(g)f]) = (\rho(g) \otimes \rho(g))\hat{K}(z, T[f]). \quad (36)$$

Comparing these two equations, we conclude that the left hand sides must coincide

$$\hat{K}(gz, \rho(g)T[f]) = \hat{K}(gz, \rho(g)T[f]) \quad (37)$$

for all  $g \in O(p, q)$ ,  $x, y \in \mathbb{R}^{p,q}$ . These expressions agree if and only if

$$\rho(g)T[f] = T[\rho(g)f] \quad (38)$$

which is the equivariance constraint on the operator  $T$ . □

### A.4. Incompleteness of CSCNNs; $O(2, 0)$

Since the grade mixing in CEGNNs happens exclusively via geometric product, it is sufficient to demonstrate how multivector encoding relative position multiplies with itself. We use sympy ((Meurer et al., 2017)) to do the algebraic manipulations. The code snippet is provided in Code 1.

```
1 import sympy
2
3 # relative position in polar coordinates
4 r, phi = sympy.symbols('r phi', real=True)
5 # embedding as a Clifford multivector
6 r_clifford = [r, r*sympy.cos(phi), r*sympy.
7               sin(phi), 0]
8 # compute geometric product and print the
9   output
10 k = geometric_product(r_clifford,
11                      r_clifford)
```

```
9 print(f"{k[0]} + {k[1]}e1 + {k[2]}e2 + {k[3]}e1e2")
10
11 # 2*r**2 + 2*r**2*cos(phi)e1 + 2*r**2*sin(phi)e2 + 0e1e2
```

Code 1. Geometric product of a multivector with itself does not propagate angular information  $\phi$ .

### A.5. Completeness of Conditional CSCNNs; $O(2, 0)$

Similarly, it can be demonstrated that for the case of two different multivectors, one of which encodes relative position, the output of the geometric product does contain angular information  $\phi$  in the scalar part, which is the necessary part to replicate the analytical solution of  $O(2)$ -steerable kernels. The code snippet is provided in Code 2. By increasing the number of layers in CEGNN, and the number of auxiliary multivector, we are able to recover high-frequency functions in the scalar part (see Code 3).

```
1 import sympy
2
3 # relative position in polar coordinates
4 r, phi = sympy.symbols('r phi', real=True)
5 # elements of the second multivector
6 x0, x1, x2, x3 = sympy.symbols('x0 x1 x2 x3', real=True)
7 # embedding as a Clifford multivector
8 r_clifford = [r, r*sympy.cos(phi), r*sympy.
9               sin(phi), 0]
10 # second multivector
11 x_clifford = [x0, x1, x2, x3]
12 # compute geometric product and print the
13   output
14 kx = geometric_product(r_clifford,
15                      x_clifford)
16
17 print(f"{kx[0]} + {kx[1]}e1 + {kx[2]}e2 + {kx[3]}e1e2")
18
19 # r*(x0 + x1*cos(phi) + x2*sin(phi)) + \
20 # r*(x0*cos(phi) + x1 - x3*sin(phi))e1 + \
21 # r*(x0*sin(phi) + x2 + x3*cos(phi))e2 + \
22 # r*(-x1*sin(phi) + x2*cos(phi) + x3)e1e2
```

Code 2. Geometric product of two multivectors allows angular information to propagate.

```
1 # elements of the third multivector
2 y0, y1, y2, y3 = sympy.symbols('y0 y1 y2 y3', real=True)
3 ky = geometric_product(r_clifford,
4                       y_clifford)
5
6 # compute geometric product between the
7   outputs (2nd layer)
8 k = geometric_product(kx, ky)
9
10 # print the scalar part of the output
11 print(kx[0])
12 # r**2*( 2*x0*y0 + x2*y2 + \
13 #        2*x0*y1*cos(phi) + 2*x0*y2*sin(phi) ) + \
```

```

12 #      2*x1*y0*cos(phi) + 2*x2*y0*sin(phi)
13 #      ) + \
14 #      x1*y1*cos(2*phi) + x1*y1 + x1*y2*
15 #      sin(2*phi) + \
16 #      x2*y1*sin(2*phi) - x2*y2*cos(2*phi)
17 #      )
18 #)
    
```

Code 3. Increasing the number of layers and auxiliary multivectors allows computing high-frequency components.

## B. Implementation details

The basic ResNet architecture that was used for constructing the Conditional-CSCNN, the CSCNN, the Clifford ResNet and the  $O(n)$ -steerable ResNet were based on the setup of (Wang et al., 2021; Brandstetter et al., 2023; Gupta & Brandstetter, 2022). They consist of 8 residual blocks with  $7 \times 7$  and  $7 \times 7 \times 7$  sized kernels for the 2D and 3D experiments respectively. We used two embedding and two output layers. When constructing the other baseline models, we closely followed the original implementations discussed in their respective papers, which are referenced in 5.2. The models have approximately 7M parameters for the Navier Stokes, 7.5M for the one step Shallow-waters and 1.5M for Maxwell’s experiments. To scale up C-CSCNN to 55M parameters in the SWE-5 task, we increased the number of residual blocks to 12, and the number of hidden channels from 48 to 96. Similarly, we increased the hidden dimensions of the parametrising kernel network from 12 to 16, and increased its depth from 4 to 6 layers.

### B.1. Conditional kernel implementation

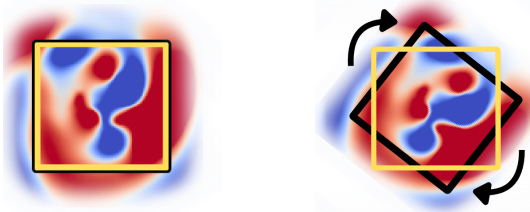


Figure 4. Illustration of how the receptive field of a finite, discretized kernel changes under rotations. The square support causes operations to break equivariance near the corners.

In constructing the Conditioned Clifford-Steerable Kernels, we built on the architecture described in Appendix A of (Zhdanov et al., 2024). We form the conditioning vector by a *masked spatial mean* computed for each channel  $c$  and blade/grade  $k$ . On a grid  $\Omega \subset \mathbb{R}^d$  and with the indicator  $\chi_{B_r}$  of the largest centered ball  $B_r$  (the *circular / spherical mask*, which we explain below), the pooled stack is

$$(T_{\text{pool}}[f_{\text{in}}])_c^{(k)} := \frac{1}{|\Omega|} \sum_{x \in \Omega} \chi_{B_r}(x) f_{\text{in},c}^{(k)}(x),$$

The resulting conditioning multivector stack is then concatenated with the relative position vector to form the input to the Kernel Network.

**Circular mask** : In practice, the multivector feature fields are often discretised as square/rectangular-shaped arrays  $(c, X_1 \dots X_d, 2^d)$ . Thus, an operation defined solely on this finite grid will break equivariance towards the corners of the domain, as shown in Figure 4. To overcome this, we apply circular masking (set grid values outside of the circle/sphere to 0) before the pooling operation, making it  $O(n)$ -equivariant in the continuum.

**Normalisation** To avoid instability in the early phases of training, we employ learnable, equivariant grade-wise normalisation of the conditioning vector stack. Note that this was only necessary for the MW2 task.

### B.2. Training details

For the training, we adopted the optimised hyperparameters from (Zhdanov et al., 2024) for our models in the NS, MW3 and MW2 experiments. For SWE-1, we adopted hyperparameters from the Transolver paper (Wu et al., 2024a). We used Adam optimizer (Kingma & Ba, 2015) with cosine learning rate scheduler for SWE with, and for MW3 and MW2 without warmup. Each model was trained to convergence. The models were trained on one node of Snellius, the Dutch national computing cluster with 1-4 NVIDIA A100 GPUs.

## C. Datasets

**2D Navier Stokes equations** The ground truth simulations are taken from (Gupta & Brandstetter, 2022) and are based on  $\Phi$ Flow by (Holl & Thuerey, 2024). From the corresponding validation and test partitions, we randomly sampled 1024 trajectories. The simulations were generated on a  $128 \times 128$  pixel grid with uniform spatial spacing  $\Delta x = \Delta y = 0.25$  m and a time step of  $\Delta t = 1.5$  s. The task consists of predicting the proceeding timestep based on the previous 4 states.

**2D Shallow-water equations** The dataset os obtained from (Gupta & Brandstetter, 2022), who utilised an implementation of `SpeedyWeather.jl` (Kl  wer et al., 2024). The simulations are performed on a grid with spatial resolution of  $192 \times 96$ ,  $\Delta x = 1.875^\circ$ ,  $\Delta y = 3.75^\circ$  and temporal resolution of  $\Delta t = 48$ h. The equations evaluated are in velocity function formulation, with vector wind velocity and scalar pressure fields to be predicted. We sampled trajectories from the validationa and test partitions randomly. For the SWE-1 task, the domain was cropped and downsampled to a grid of  $64 \times 64$  spatial points to speed up training. The

task for SWE-1 is to predict the next timestep given the previous 4, whereas for SWE-5, the proceeding 5 states are to be predicted, given the previous 2. The final dataset for SWE-1 consisted of 512 test, 512 valid, and max 2048 train trajectories. For SWE-5, the entire train, test, valid dataset, described in (Gupta & Brandstetter, 2022) was used.

**3D Maxwell’s equations** Within the non-relativistic  $\text{Cl}(\mathbb{R}^{3,0})$  setting, we represent the electric field  $E$  as a vector field and the magnetic field  $B$  as a bivector field. The dataset, drawn from (Brandstetter et al., 2023), consists of 3D Maxwell simulations discretized on a  $32 \times 32 \times 32$  voxel grid with uniform spacing  $\Delta x = \Delta y = \Delta z = 5 \times 10^{-7}$  m and time step  $\Delta t = 50$  s. The validation and test splits each contain 128 simulations. Similarly to NS2, the task consists of predicting the proceeding timestep based on the previous 4 states.

**2D relativistic Maxwell’s equations** We generate a dataset for Maxwell’s equations in 2+1D spacetime ( $\mathbb{R}^{1,2}$ ) utilizing the PyCharge simulation package by (Filipovich & Hughes, 2022). The simulations model the dynamics of electromagnetic fields emitted by oscillating and orbiting point charges moving at relativistic speeds. The spacetime grid is discretized with a resolution of  $128 \times 128$  points, corresponding to a spatial extent of 50 nm and a temporal duration of  $3.77 \cdot 10^{-14}$  s.

Each simulation is initialized with a unique configuration of charge sources, governed by the following randomly sampled parameters: **Source Composition:** A combination of 2 to 4 oscillating charges and 1 to 2 orbiting charges, with integer magnitudes sampled uniformly from the range  $[-3e, 3e]$ . **Initial Conditions:** Sources are placed uniformly on the grid with a predefined minimum separation. Each is assigned a random linear velocity and either oscillates in a random direction or orbits with a random radius. **Relativistic Constraint:** Oscillation/rotation frequencies and velocities are sampled such that the total particle velocity does not exceed  $0.85c$ , a necessary constraint to ensure the stability of the PyCharge solver.

To handle the wide dynamic range of the resulting field strengths, we apply a normalization scheme. The generated field bivectors are divided by their Minkowski norm and then multiplied by the logarithm of that norm. Although Minkowski norms can be zero or negative, we found they were consistently positive in our generated data. Finally, we filter numerical artifacts by removing any outlier simulations that exhibit a standard deviation greater than 20. The curated dataset is split into 2048 training, 256 validation, and 256 test simulations.

## D. More Experimental Results

We provide a comprehensive overview of the experiment results in Table 4. By transforming consistently under the symmetries of the physical system, Conditional CSCNNs are able to effectively capture the true governing dynamics of fluid flow, providing accuracy improvements even in highly transient regions of the simulation domain, such as is visualised in Figure 5.

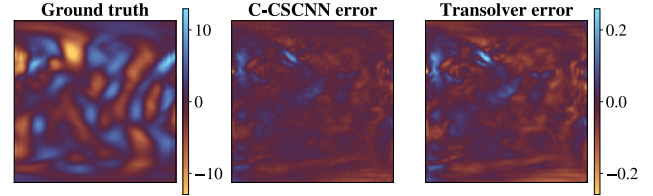


Figure 5. Signed residuals (prediction—ground truth) of wind velocity vector component  $u$  in the one step ahead predictions for Shallow-Water Equations  $\mathbb{R}^2$ .

### D.1. Ablation study of conditional operator

We study the effect of choosing different conditional operators on the 2D Shallow-Water (SWE-1) task using  $N = 256$  training trajectories. Specifically, we compare (i) the unconditioned CSCNN baseline, (ii) C-CSCNN with mean-pooling-based conditioning, (iii) C-CSCNN with max-pooling-based conditioning, and (iv) C-CSCNN with a fixed randomly sampled multivector (drawn from a standard normal distribution) used as a condition. The mean- and max-pooling variants preserve equivariance of the conditioning signal, whereas the fixed random condition does not.

Table 3. Ablation of conditioning operator choices on SWE-1 with  $N = 256$  trajectories. Metric: MSE (lower is better). Best result is in bold while the second best is underlined.

| Model / Conditioning       | MSE           |
|----------------------------|---------------|
| CSCNN (no conditioning)    | 0.5699        |
| C-CSCNN (mean pooling)     | <b>0.2982</b> |
| C-CSCNN (max pooling)      | <u>0.3097</u> |
| C-CSCNN (random condition) | 0.3225        |

Across these variants, mean pooling yields the lowest error, with max pooling a close second. Notably, both mean and max pooling preserve equivariance of the conditioning signal, whereas the fixed random condition does not. The performance gap between these settings highlights the practical benefit of equivariant conditioning in this setting and supports the design choice underlying our framework.

| NS                          | $N=64$                                | $N=512$        | $N=2048$       | $N=4096$       |
|-----------------------------|---------------------------------------|----------------|----------------|----------------|
| C-CSCNN (Ours)              | <b>2.38e-3</b>                        | <b>5.80e-4</b> | <b>2.70e-4</b> | <b>1.80e-4</b> |
| Transolver                  | <u>2.49e-3</u>                        | <u>7.10e-4</u> | <u>4.10e-4</u> | <u>3.40e-4</u> |
| CSCNN                       | 2.63e-3                               | 9.00e-4        | 5.30e-4        | 4.50e-4        |
| Clifford ResNet             | 4.21e-3                               | 2.01e-3        | 1.18e-3        | 1.11e-3        |
| Steerable ResNet            | 8.41e-3                               | 2.30e-3        | 1.35e-3        | 1.25e-3        |
| Cl.-St. ResNet              | 2.62e-3                               | 1.21e-3        | 9.14e-4        | 8.80e-4        |
| ResNet                      | 4.57e-3                               | 3.29e-3        | 2.80e-3        | 2.79e-3        |
| CViT                        | 1.21e-2                               | 2.15e-3        | 1.01e-3        | 9.45e-4        |
| G-FNO                       | 1.87e-2                               | 6.77e-3        | 5.41e-3        | 4.90e-3        |
| FNO                         | 3.17e-2                               | 1.16e-2        | 8.70e-3        | 8.20e-3        |
| SWE-1                       | $N=64$                                | $N=256$        | $N=1024$       | $N=2048$       |
| C-CSCNN (Ours)              | <b>1.389</b>                          | <b>0.314</b>   | <b>0.073</b>   | <b>0.052</b>   |
| Transolver                  | <u>1.513</u>                          | <u>0.452</u>   | <u>0.090</u>   | <u>0.059</u>   |
| CSCNN                       | 1.783                                 | 0.570          | 0.143          | 0.113          |
| CViT                        | 1.987                                 | 0.713          | 0.197          | 0.126          |
| Swin-Tr.                    | 2.134                                 | 0.820          | 0.207          | 0.150          |
| U-Net                       | 8.539                                 | 3.852          | 1.316          | 0.606          |
| LSM                         | 9.187                                 | 4.328          | 1.419          | 0.809          |
| MW3                         | $N=64$                                | $N=256$        | $N=512$        |                |
| C-CSCNN (Ours)              | <u>6.07e-4</u>                        | <u>5.44e-4</u> | <u>4.51e-4</u> |                |
| Transolver                  | <b>4.96e-4</b>                        | <b>4.24e-4</b> | <b>3.99e-4</b> |                |
| CViT                        | 1.28e-3                               | 1.08e-3        | 9.65e-4        |                |
| CSCNN                       | 7.50e-4                               | 7.02e-4        | 6.82e-4        |                |
| ResNet                      | 3.34e-3                               | 1.90e-3        | 1.67e-3        |                |
| Clifford ResNet             | 3.83e-3                               | 2.57e-3        | 1.70e-3        |                |
| FNO                         | 1.91e-2                               | 1.66e-2        | 1.41e-2        |                |
| MW2                         | $N=512$                               | $N=1024$       | $N=2048$       |                |
| C-CSCNN (Ours)              | <u>0.327</u>                          | <u>0.293</u>   | <u>0.252</u>   |                |
| Transolver                  | <b>0.263</b>                          | <b>0.221</b>   | <b>0.170</b>   |                |
| CViT                        | 0.660                                 | 0.486          | 0.453          |                |
| CSCNN                       | 0.389                                 | 0.368          | 0.349          |                |
| ResNet                      | 0.808                                 | 0.766          | 0.665          |                |
| SWE-5                       | <i>Rel. <math>L^2</math> (5-step)</i> |                |                |                |
| CViT-L (92M)                | <b>1.56%</b>                          |                |                |                |
| U-F2Net (344M)              | <u>1.89%</u>                          |                |                |                |
| CViT-B (30M)                | 2.69%                                 |                |                |                |
| C-CSCNN (55M)               | 2.94%                                 |                |                |                |
| C-CSCNN (10M)               | 3.51%                                 |                |                |                |
| UNO (440M)                  | 3.79%                                 |                |                |                |
| FNO (268M)                  | 3.97%                                 |                |                |                |
| CViT-S (13M)                | 4.47%                                 |                |                |                |
| U-Net <sub>att</sub> (148M) | 5.68%                                 |                |                |                |
| DilResNet (4.2M)            | 13.20%                                |                |                |                |

Table 4. MSE across experiments and training sizes ( $N$ ). Rel.  $L^2$  reported for SWE-5. On SWE-5, the parameter counts of each model is listed in parantheses next the models name, while for the other tasks, all models are parameter matched as discussed in Appendix B. Best per column is in bold and the second-best per column is underlined.

## D.2. Inference time

We report inference times on the Maxwell 3D task using a single A100 GPU, averaged over the test set. The results are shown in Table 5.

Table 5. Inference time on the Maxwell 3D task using a single A100 GPU, averaged over the test set.

| Model          | Time (ms) |
|----------------|-----------|
| Simple ResNet  | 1.80      |
| Transolver     | 39.06     |
| CSCNN          | 53.83     |
| C-CSCNN (Ours) | 54.30     |

Two observations are worth highlighting. First, the mean conditioning mechanism adds virtually no overhead: C-CSCNN is only (0.87%) slower than the unconditioned CSCNN. Second, our current implementation of Clifford algebra operations, including geometric products and multivector processing, is not optimised for speed. However, hardware-efficient implementations such as Flash Clifford (Zhdanov, 2025) already exist and can be integrated to significantly reduce this gap.